PRODUCT REQUIREMENTS DOCUMENT / VERSION 1.0

Smoooth

Infrastructure freedom, built for AI agents.

The infrastructure platform that lets coding agents choose, provision, connect, and operate the technologies an application needs—without forcing every project into the same stack.

> **Core idea: Supabase-level convenience across the software stack, with the freedom to choose the technologies that fit the product.**

DOCUMENT	STATUS	PRODUCT THESIS
Product definition and MVP direction	Working PRD · October 2026	Agent-first infrastructure orchestration across providers and technologies

Contents

01 Executive summary

02 Problem and opportunity

03 Product vision and principles

04 Target users and use cases

05 Product experience

06 Functional requirements

07 MCP and agent interface

08 Infrastructure capability model

09 Security, control, and reliability

10 MVP scope and rollout

11 Architecture direction

12 Success metrics and validation

13 Risks and open decisions

14 Acceptance criteria

01. Executive summary

Smoooth is an agent-first infrastructure platform. It gives AI coding agents a consistent way to discover, provision, configure, connect, deploy, observe, and operate the infrastructure a software product needs.

The central premise is that the next generation of software will increasingly be built by agents, but agents still face fragmented infrastructure workflows. They may be able to write an application, yet provisioning the right database, configuring networking and credentials, deploying supporting services, and keeping the whole system healthy requires provider-specific knowledge and manual glue.

> **Smoooth should let an agent choose the appropriate stack for a user's requirements, then make that stack practical to build and operate.**

The product takes the convenience associated with platforms such as Supabase and extends it across a broader range of technologies and deployment models. Smoooth is not a Supabase clone, a Docker dashboard, or a self-hosting product. Those are reference points or use cases. The core product is a reliable infrastructure capability layer designed for agents and governed by users.

Product promise

> **Tell your agent what you want to build. Smoooth helps it choose, provision, connect, and operate the infrastructure needed to make it real.**

Product outcome

- Developers can use the stack that best fits their product rather than being forced into one backend or hosting model.
- Agents can perform infrastructure work through structured tools instead of improvising through shell commands and fragmented APIs.
- Users retain control over budgets, permissions, providers, data, and risky actions.
- Infrastructure operations become inspectable, repeatable, and recoverable.

Strategic distinction

Smoooth should not impose a small, permanent list of approved technologies as a product philosophy. It should aim for broad compatibility through a capability and provider model. However, each supported capability must have a clear, honest support level. Broad reach is the direction; reliable lifecycle management is the standard.

02. Problem and opportunity

The problem

AI coding agents are increasingly capable of producing application code. Turning that code into a working product still requires choices and operations across databases, caches, queues, object storage, authentication, containers, networking, deployment targets, secrets, observability, backups, and recovery.

- Fragmented interfaces: each provider and tool has its own API, concepts, permissions, and failure modes.
- Default-stack bias: agents often select familiar managed platforms because they are easy to integrate, even when another architecture may fit better.

- Operational gaps: creating a service is not enough; it must be connected securely, monitored, backed up, upgraded, and recovered.
- Human glue: developers frequently bridge gaps with shell scripts, provider dashboards, configuration files, and undocumented operational knowledge.
- Lock-in by convenience: a convenient default can become a constraint when the project needs a different database, runtime, hosting model, or topology.

The opportunity

Smoooth can become the common operational interface between coding agents and heterogeneous infrastructure. Instead of requiring every agent, framework, or application to learn every provider independently, Smoooth offers consistent tools for infrastructure capabilities while preserving the underlying technologies and user choice.

Evidence versus hypothesis

Statement	Status	Implication
Developers pay for reduced backend and operations work.	**Established market signal**	Convenience is a credible value proposition.
Agents can operate infrastructure through tools and APIs.	**Technically feasible**	A structured agent interface is practical.
A unified agent-first layer will be preferred over direct provider integrations.	**Hypothesis to validate**	Test actual workflows and willingness to pay.
Users want broad technology freedom without more operational burden.	**Core product hypothesis**	Validate with diverse application builds.

This PRD treats the broad opportunity as the product thesis, not as proven customer demand. The MVP must validate whether Smoooth materially reduces the time, errors, and human intervention required to get an agent-built application running.

03. Product vision and principles

Vision

> **Any coding agent should be able to build with the infrastructure an application needs—not only the infrastructure one platform happens to offer.**

Product principles

Principle	What it means in practice
Agent-first	Agents are primary operators, not secondary users of a dashboard. Every core workflow must be available through structured, documented tools.
User-directed freedom	The agent may recommend technologies, but user policies govern approved providers, budgets, data location, and high-impact actions.
Convenience over complexity	Smoooth handles provisioning, wiring, configuration, verification, and lifecycle tasks—not merely resource creation.
Technology-neutral core	Core concepts describe capabilities and lifecycle operations rather than hard-coding the product around one database or hosting vendor.
Honest support	Clearly distinguish fully managed, self-hosted, provider-integrated, experimental, and unsupported capabilities.
Composable by design	Applications may combine technologies across providers and deployment models when appropriate.
Safe autonomy	Agents can act autonomously within explicit policies, with approval gates for destructive, expensive, or irreversible operations.
Evidence, not assertions	Smoooth verifies deployment and service health and returns actionable evidence instead of reporting success merely because a command completed.

What Smoooth is

- An infrastructure capability layer and operational control plane for coding agents.
- A consistent MCP/API/CLI surface for provisioning and managing diverse technologies.
- A system that manages relationships between application code, services, credentials, networking, deployments, and operational state.
- A way to choose between managed services, customer-owned infrastructure, and other supported execution environments.

What Smoooth is not

- Not limited to self-hosting or to customer-owned VPS instances.
- Not a requirement that every customer use the same database or backend architecture.
- Not simply an MCP wrapper around unrelated provider APIs.
- Not a promise that every technology will be fully managed from day one.
- Not an unrestricted shell or root-access agent.

04. Target users and use cases

Primary users

User	Need	Success looks like
AI-assisted builders	Ship applications from natural-language requirements and generated code.	Agent provisions a suitable stack and delivers a verified running app.
Indie developers and founders	Avoid spending days on infrastructure and DevOps.	Less setup time, fewer dashboards, and clear ongoing costs.
Small engineering teams	Use the right mix of technologies without maintaining one-off automation for each.	Repeatable provisioning and operations across projects.
Agencies and consultants	Deploy and maintain varied client stacks reliably.	Reusable workflows, scoped access, auditability, and predictable operations.

Core use cases

Use case A — Build a new product

A user asks an agent to build a product with particular requirements. The agent proposes a stack, identifies infrastructure needs, checks the user's Smoooth policies, provisions services, configures connections, deploys the application, and validates the working system.

Use case B — Choose a non-default architecture

An agent determines that MongoDB, Redis, and a Docker-based worker service are a better fit than a default managed Postgres backend. It can choose that architecture when allowed, explain the trade-offs, and use Smoooth to operate it.

Use case C — Bring an existing app

A developer connects a repository or existing environment. Smoooth inspects the app, identifies required services, produces a deployment plan, and helps execute it without assuming the app must be rewritten for a particular platform.

Use case D — Operate a production system

An agent checks health, reviews deployment events and logs, diagnoses a failed release, recommends a safe fix, and executes the permitted recovery action. It returns evidence and records what changed.

Use case E — Mix hosting models

A product may use a managed database, a self-hosted worker, object storage from another provider, and a container deployment on a customer-owned server. Smoooth should represent the dependencies and access boundaries explicitly.

05. Product experience

The primary workflow

Step	Agent action	Smoooth responsibility
1. Understand	Collects requirements and constraints.	Expose capabilities, policies, provider options, pricing estimates, and existing resources.
2. Plan	Chooses and explains a proposed architecture.	Validate compatibility, required permissions, costs, and dependencies; produce a plan.
3. Approve	Requests approval where required.	Enforce policy and approval rules; bind approval to the exact plan version.
4. Provision	Requests services and configuration.	Create resources idempotently, track progress, and report actionable errors.
5. Connect	Requests app-to-service wiring.	Manage scoped credentials, network access, configuration, and dependency relationships.
6. Deploy	Builds and releases the application.	Run supported deployment workflows and preserve release history.
7. Verify	Checks that the product works.	Run health checks and return evidence of readiness and connectivity.
8. Operate	Monitors and manages the system.	Provide logs, metrics, events, backups, updates, and safe recovery actions.

Experience requirements

- The agent should discover available capabilities and support levels before committing to a stack.
- The plan should show resources to be created, costs where available, permissions, network exposure, data location, and destructive effects.
- Provisioning and deployment should be asynchronous where needed, with stable operation IDs and status polling or events.
- Failures should identify the failed stage, likely cause, relevant redacted evidence, and safe next actions.
- A user should be able to inspect the same plan, execution history, and system state in the dashboard.
- The dashboard should complement the agent workflow rather than being required for every routine task.

06. Functional requirements

P0 — Required for a credible first release

Capability	Requirements	Acceptance signal
Workspace and identity	Users, workspaces, memberships, scoped tokens, revocation, role-based permissions.	An agent can only access resources authorized for its workspace and token.
Capability discovery	List supported providers, service types, operations, constraints, and support levels.	An agent can discover what it can provision and manage without hard-coded assumptions.
Infrastructure inventory	List and inspect resources, their status, dependencies, region, ownership, and available operations.	Agent and dashboard agree on resource state.
Planning and policy	Generate a typed plan with dependencies, estimated cost where possible, permission requirements, and risk classification.	Plan validation catches unsupported configurations and policy violations before execution.
Provisioning orchestration	Start, track, retry, and reconcile operations; use idempotency keys and durable state.	Retrying a request does not create duplicate resources.
Application deployment	Deploy a repository or containerized application to at least one supported target; configure environment and service connections.	A supported sample app reaches a verified healthy state.
One complete data path	Support at least one database service end-to-end, including provision, credentials, connection, backup policy or documented limitation, and deletion safeguards.	The deployed app can securely read/write data and recover from a failed deployment.
Observability	Expose operation history, status, redacted logs, service health, and structured errors.	An agent can diagnose a seeded failure without relying on an unstructured shell session.
Lifecycle actions	Restart, redeploy, inspect, and stop supported resources; provide rollback where the runtime supports it.	Actions are authorized, auditable, and verifiable.
MCP server	Typed tools for discovery, planning, provisioning, inspection, deployment, and operations.	An external coding agent completes the primary workflow using MCP.
Dashboard	Inventory, plans, approvals, operation detail, logs, credentials metadata, and audit history.	A human can understand what the agent did and intervene when needed.

P1 — Expand capability coverage

- Additional databases and service types, prioritized by user requests and real application tests.
- More execution targets: customer-owned VPS, managed compute, and selected cloud/provider services.
- Automated backups and tested restore workflows for supported stateful services.
- Push-triggered deployments, preview environments, team roles, notifications, and reusable templates.
- Provider cost visibility, resource budgets, usage limits, and policy-based approval thresholds.
- Scheduled maintenance, safe updates, drift detection, and guided recovery.

P2 — Platform expansion

- Kubernetes lifecycle and workload operations where there is a validated customer need.
- Broader service catalog, queues, object storage, search, vector databases, and specialized runtimes.
- Multi-region and multi-provider topologies, advanced networking, and high-availability patterns.
- Organization-wide policies, compliance exports, advanced audit, and enterprise identity integrations.
- Marketplace or community-contributed capability adapters with conformance tests.

These priority labels are sequencing guidance, not a permanent limit on the vision. Add capabilities based on user demand, agent workflow value, implementation feasibility, operational burden, and safety—not merely the number of integrations.

07. MCP and agent interface

MCP is a primary product interface, not a feature added after the dashboard. It should make Smoooth discoverable and usable by coding agents while remaining consistent with the public API and internal operation model.

Tool design rules

- Use typed, task-oriented tools with validated inputs and structured outputs.
- Expose discovery before mutation: agents should inspect capabilities and constraints first.
- Separate planning from execution so policies and approvals can apply to a concrete plan.
- Return operation IDs for long-running tasks and provide status, logs, events, and final evidence.
- Use idempotency keys and safe retry semantics for mutations.
- Enforce authorization and policy server-side; never rely on the agent to self-police.
- Do not provide a general-purpose unrestricted shell tool as the core abstraction.

Initial MCP tool groups

Group	Representative tools
Discover	list_capabilities, list_providers, list_service_types, get_capability_schema
Inspect	list_resources, inspect_resource, inspect_application, inspect_dependencies
Plan	propose_architecture, create_plan, validate_plan, estimate_cost, explain_plan_risks
Provision	execute_plan, get_operation, cancel_operation, reconcile_resource
Connect	create_service_binding, inspect_network_access, rotate_credential, validate_connection
Deploy	inspect_repository, build_application, deploy_release, get_deployment_status, rollback_release
Operate	get_health, get_logs, get_metrics, restart_service, update_service, inspect_backup, restore_backup
Govern	get_policy, request_approval, inspect_audit_events, list_costs

Structured outputs

Tool responses should include a stable resource or operation ID, current status, timestamps, changed resources, relevant constraints, redacted evidence, warnings, and next valid actions. Errors should be machine-readable with stable codes, retryability, and remediation hints.

MCP security model

Use short-lived or revocable credentials, scoped workspace permissions, explicit tool authorization, server-side policy checks, and audit events for every mutation. High-impact operations such as deleting persistent data, exposing a database publicly, or restoring over production data should require a policy-defined approval.

08. Infrastructure capability model

The long-term product cannot depend on every provider exposing identical features. Smoooth should normalize common operations while preserving provider-specific capabilities when they matter.

Capability layers

Layer	Purpose	Example
Capability schema	Describes a type of resource and its valid configuration.	Database, container workload, object store, queue, Kubernetes cluster.
Provider adapter	Maps normalized operations to provider APIs, CLIs, or local runtimes.	Managed database provider, Docker Engine, cloud API.
Lifecycle engine	Coordinates create, inspect, update, delete, retry, reconcile, and recovery.	Durable state machine with idempotent operations.
Connection and identity	Handles credentials, network rules, endpoints, and service bindings.	App receives a scoped database URL without exposing it in logs.
Policy engine	Enforces permissions, budgets, regions, and approval requirements.	Block a destructive action without explicit approval.
Evidence and audit	Records what changed and verifies outcomes.	Health checks, operation events, resource diffs, and audit trail.

Support levels

Level	Meaning
Managed	Smoooth owns or directly manages the lifecycle and documented operational guarantees.
Integrated	Smoooth operates a third-party provider through an adapter; provider-specific limits remain.
Self-hosted	Smoooth operates software on infrastructure controlled by the customer.
Experimental	Usable for evaluation, with explicit limitations and no implied production guarantee.
Unsupported	Not discoverable as a ready-to-use capability; agents may still use external tools outside Smoooth's managed guarantees.

This model allows the platform to be broad without pretending all services have identical reliability, pricing, backup behavior, or feature parity.

09. Security, control, and reliability

Required controls

- Tenant isolation: every resource lookup and mutation is scoped and authorized on the server.

- Secrets: encrypted at rest, tightly scoped, redacted from logs and tool output, and never included in build contexts unless explicitly required.
- Least privilege: provider credentials and service bindings grant only the permissions required for the intended operation.
- Plan-bound approval: approvals refer to a specific plan version or hash; material changes require revalidation.
- Cost controls: allow user-defined budgets, resource ceilings, provider allowlists, and approval thresholds.
- Network safety: prevent unsafe public exposure by default; validate endpoints and block SSRF paths in fetch and inspection workflows.
- State protection: destructive data operations require explicit safeguards; application rollback must not silently roll back or delete persistent data.
- Auditability: record actor, token identity, action, target, plan, outcome, timestamp, and redacted evidence.
- Supply-chain safety: treat repositories, images, packages, startup scripts, and agent-generated configuration as untrusted inputs.
- Recovery: document which failures can be retried, rolled back, restored, or require human intervention.

Autonomy model

Risk tier	Example	Default behavior
Low	Inspect status, read logs, validate configuration.	Allow when authorized.
Moderate	Create a low-cost development resource, restart a service.	Allow within policy; notify and audit.
High	Increase spend materially, expose a service publicly, change production networking.	Require explicit policy authorization; approval may be required.
Critical	Delete production data, destroy a cluster, restore over live data, rotate a root credential.	Require explicit approval and strong confirmation; provide a clear impact summary.

The aim is not to constrain agents arbitrarily. It is to let them act with confidence inside boundaries chosen by the user, with more freedom available to users who explicitly configure it.

10. MVP scope and rollout

MVP objective

> **Prove that an external coding agent can independently take a real application from requirements to a healthy, connected, observable deployment through Smoooth—with the user controlling policy and risk.**

Choose one complete vertical slice

Do not start by implementing a thin version of every database, cloud, and orchestration technology. Start with a complete end-to-end workflow and a capability architecture that can expand. The first stack is a proving ground, not a statement that Smoooth will always be limited to it.

MVP element	Recommended starting scope	Why
Agent interface	MCP server plus stable API	Validates the primary user and operating model.
Application runtime	Docker-based deployment on one practical target	Proves build, network, release, logs, and lifecycle operations.
Database	One production-credible database path, with provisioning and secure connection	Proves the app-to-service relationship rather than only container startup.
Control plane	Workspace, resource inventory, plan execution, durable operations, audit	Creates the foundation for additional providers and service types.
Human interface	Small dashboard for visibility, policy, approvals, and recovery	Lets users inspect and control what agents do.
Verification	Health checks, connection test, release history, actionable errors	Demonstrates that operations actually succeeded.

Suggested delivery phases

Phase	Deliverable	Exit criteria
0. Discovery	Interview builders; test 3–5 representative application scenarios; map the most painful infrastructure steps.	Clear initial customer, repeatable use case, and chosen vertical slice.
1. Core platform	Identity, workspaces, resource model, capability discovery, policies, durable operation engine, audit.	Authorized, idempotent operations work end-to-end in a test environment.
2. First infrastructure path	One runtime target, one database path, secure bindings, logs, health checks, rollback/recovery boundaries.	A sample app is deployed and its database connection is verified.
3. Agent workflow	MCP tools for discovery, planning, provisioning, deployment, inspection, and operations.	A supported coding agent completes the workflow without manual dashboard setup.
4. Design partners	Run real user projects; capture failures, interventions, time-to-working-app, and cost.	Repeated successful workflows and evidence of willingness to pay.
5. Expand coverage	Prioritize next providers and technologies from observed demand and failure patterns.	Each added capability passes conformance, security, and lifecycle tests.

Explicitly not required for MVP

- Every database, cloud provider, container orchestrator, and storage system.
- A complete Supabase feature set such as a universal auth platform, realtime system, and every adjacent service.
- A complete Kubernetes management platform before customer evidence requires it.
- A large template catalog, marketplace, or elaborate visual workflow builder.
- Autonomous execution of every destructive action without policy or approval.
- A polished dashboard that duplicates every action the agent can perform.

11. Architecture direction

The architecture should separate the stable agent-facing product contract from provider-specific execution. This reduces the risk of building a collection of one-off integrations that cannot be operated consistently.

Logical components

Component	Responsibility
MCP server and public API	Expose versioned, typed operations to agents, CLI clients, and other integrations.
Control plane	Identity, workspaces, policies, resource inventory, planning, approvals, and operation coordination.
Durable operation engine	Queue and track long-running tasks, retry safely, reconcile drift, and preserve event history.
Capability registry	Describe supported resource types, schemas, lifecycle operations, provider constraints, and support levels.
Provider adapters	Translate normalized operations into provider APIs, container runtimes, or customer-side agents.
Execution workers	Perform builds and infrastructure operations in isolated, constrained environments.
Secrets and identity service	Issue, encrypt, scope, rotate, and audit credentials and service bindings.
Evidence and observability	Collect operation events, health checks, logs, metrics, and verification results.
Dashboard	Show system state, plans, costs, approvals, failures, and recovery options.

Architecture requirements

- Long-running operations must be durable across worker restarts and support reconciliation after partial failure.
- Every adapter must declare which lifecycle operations it supports and how it handles idempotency, retries, deletion, backups, and recovery.
- The control plane should not need provider-specific business logic scattered across the dashboard or MCP server.
- Use a typed resource and dependency model, but allow provider-specific configuration and outputs where normalization would lose important capability.
- Version capability schemas and MCP/API contracts; document compatibility and deprecation.
- Keep credentials and execution close to the least-privileged component that needs them.
- Test adapter conformance with standard lifecycle, security, and failure scenarios before calling a capability production-ready.

Technology choices

This PRD intentionally does not lock the team into a final programming language, database, queue, or deployment topology. The product contract and safety properties matter more than selecting fashionable infrastructure early. Choose implementation technologies after reviewing expected operation volume, execution isolation needs, team capability, reliability requirements, and deployment constraints.

12. Success metrics and validation

North-star metric

> **Verified agent-completed infrastructure workflows per active workspace per week.**

A workflow counts only when the requested application or infrastructure change reaches its intended verified state, with no unplanned human intervention. Track assisted workflows separately from autonomous completion.

Supporting metrics

Metric	Definition / purpose
Time to first working app	Elapsed time from connecting a repository or stating a goal to verified application readiness.
Agent completion rate	Percentage of supported workflows completed without unplanned human intervention.
Intervention rate	Human actions required per workflow, classified by reason.
Provisioning success rate	Percentage of operations that reach the intended state, including recovery from transient errors.
Recovery success rate	Percentage of seeded or real failures resolved through documented automated or guided recovery.
Technology-choice coverage	Share of requested architectures that Smoooth can support end-to-end, not merely create a resource for.
Cost transparency	Share of provisioning plans with a useful estimate or explicit statement that cost cannot be estimated.
Retention and usage	Repeat weekly use, number of active projects, and recurring infrastructure operations.
Willingness to pay	Conversion to paid usage, accepted pricing, and expansion within teams.

Validation plan

- Recruit developers who use coding agents and have shipped at least one real application.
- Test several contrasting architectures, including a conventional managed database stack and a container-based stack with multiple services.
- Compare time, failure rate, and human interventions against the user's existing workflow.
- Ask participants to pay for continued use or commit to a paid pilot; compliments alone do not validate demand.
- Record where agents get stuck: architecture selection, missing capability, credential setup, provider limitations, unclear errors, or unsafe operations.
- Use failures to prioritize capability improvements rather than adding integrations based only on popularity.

13. Risks and open decisions

Risk / question	Why it matters	Recommended approach
Scope explosion	The space includes many providers, technologies, and operational edge cases.	Build a reusable capability architecture and expand one complete workflow at a time.
Inconsistent provider semantics	Backup, networking, deletion, scaling, and recovery differ by provider.	Normalize common actions; expose provider-specific capabilities and limitations honestly.
Agent chooses a poor architecture	More choice can increase complexity and cost.	Require rationale and trade-offs in plans; let users set preferences and constraints.
MCP security exposure	An agent-facing interface can trigger consequential operations.	Server-side authorization, scoped tokens, plan-bound approvals, audit, and risk-tiered actions.
Unreliable automation	A successful API response may not mean the system works.	Verify end state, use durable operations, and return evidence.
Operational cost	Running or supporting stateful services can create support and infrastructure burden.	Separate managed versus integrated responsibilities and model costs early.
Unclear first customer	A universal platform message may be too broad for early adoption.	Keep the vision broad while selecting a narrow, painful first workflow and audience.

Decisions to settle during discovery

- Which first user segment has the most painful, frequent infrastructure handoff from agent-generated code?
- Should the first execution target be customer-owned VPS, a managed environment, or a provider integration? Decide based on the chosen workflow and operational burden.
- Which database and runtime combination proves the complete value proposition with the least external dependency?
- What is the first pricing unit: active resource, compute usage, operation volume, or a platform subscription?
- Which operations can run autonomously by default, and which require user policy or approval?
- What does Smoooth guarantee for each support level, especially for backups, data durability, and incident response?

14. Acceptance criteria

The first release is ready for a design-partner beta when all of the following are demonstrated in a repeatable test environment:

- An external coding agent can discover Smoooth capabilities through MCP without hard-coded provider assumptions.
- The agent can generate a typed plan for a supported application architecture and receive clear validation errors for unsupported or unsafe choices.
- The user can configure policy for permissions, budgets, and high-impact operations.
- The agent can execute the approved plan, track durable progress, and safely retry transient failures.
- Smoooth can provision a supported runtime and database, establish a secure application connection, and verify that the application can use the database.

- The agent can inspect service state, retrieve relevant redacted logs, identify a seeded failure, and execute an allowed recovery action.
- Every mutation is authorized and recorded in an audit trail with a clear actor and outcome.
- The dashboard accurately reflects resources, plans, operations, and failures.
- Deleting or redeploying an application does not silently delete persistent data.
- A design partner can complete the core workflow repeatedly and expresses concrete willingness to pay or continue a paid pilot.

Definition of done for every capability

- Documented schema and supported lifecycle operations.
- Authentication, authorization, secret handling, and network exposure reviewed.
- Create, inspect, update, failure, retry, delete, and recovery behavior tested where applicable.
- Structured errors and actionable evidence returned to the agent.
- Cost and operational responsibility disclosed where relevant.
- Provider-specific limitations and support level documented.
- Conformance tests pass before production support is claimed.

> **Final product test: Can an agent build and operate the architecture the user actually needs, with less infrastructure work and fewer mistakes than the current alternatives?**